Architectural Interview Questions and Answers

Este documento contiene un banco de **60 preguntas de opción múltiple** sobre arquitectura de software, organizadas en niveles **Básico**, **Intermedio** y **Avanzado**, pensado para la preparación de entrevistas técnicas de Arquitecto de Software / Ingeniero Senior.

Cada pregunta incluye 4 opciones de respuesta, la respuesta correcta y una explicación detallada del concepto evaluado. Las respuestas correctas están distribuidas equitativamente entre las opciones a, b, c y d (15 cada una) para evitar patrones predecibles.

Índice

- [Básico: Fundamentos de Arquitectura de Software](#básico-fundamentos-de-arquitectura-de-software) (preguntas 1-20)
- [Intermedio: Estilos y Patrones Arquitectónicos](#intermedio-estilos-y-patrones-arquitectónicos) (preguntas 21-40)
- [Avanzado: Arquitectura Avanzada y Sistemas Distribuidos](#avanzado-arquitectura-avanzada-y-sistemas-distribuidos) (preguntas 41-60)

Básico: Fundamentos de Arquitectura de Software

1. ¿Qué es la arquitectura de software?

- a) Un diagrama UML que documenta las clases del sistema y sus relaciones de herencia. - b) El código fuente final que implementa los casos de uso del sistema. - c) El conjunto de decisiones estructurales fundamentales sobre cómo se organiza un sistema: sus componentes, las relaciones entre ellos y los principios que guían su diseño y evolución. - d) El conjunto de pruebas unitarias que garantizan la calidad del software. - **Respuesta:** (c) - **Explicación:** La arquitectura de software es el nivel más alto de abstracción en el diseño de un sistema: define los componentes principales, cómo se comunican entre sí, y las decisiones difíciles de cambiar más adelante (estilo, tecnología base, límites de los módulos). No es un artefacto (como un

diagrama) ni el código en sí, sino el conjunto de decisiones y restricciones que condicionan cómo se construye, opera y evoluciona el sistema. Estas decisiones determinan atributos de calidad como el rendimiento, la escalabilidad y la mantenibilidad, por lo que suelen tomarse de forma deliberada y documentarse (por ejemplo, mediante ADRs) para que el equipo entienda el 'por qué' detrás de la estructura elegida.

1. ¿Cuál es la diferencia principal entre arquitectura y diseño de software?

- a) No existe diferencia real; ambos términos son sinónimos y se usan indistintamente en la industria. - b) La arquitectura se enfoca en decisiones estructurales de alto nivel y difíciles de revertir, mientras que el diseño se enfoca en la implementación detallada de componentes individuales. - c) El diseño siempre ocurre antes que la arquitectura en el ciclo de vida del proyecto. - d) La arquitectura solo aplica a sistemas monolíticos, mientras que el diseño aplica a microservicios. - **Respuesta:** (b) - **Explicación:** Aunque no hay una línea absoluta que las separe, la distinción práctica más útil es de alcance y costo de cambio: la arquitectura trata decisiones globales (estilo del sistema, límites entre módulos, tecnologías clave, atributos de calidad) que afectan a todo el sistema y son costosas de revertir. El diseño detallado, en cambio, se ocupa de cómo se implementa una clase, un módulo o un componente específico dentro de esos límites ya establecidos, con decisiones más locales y reversibles. Grady Booch resumió esto diciendo que la arquitectura representa las decisiones significativas que son costosas de cambiar.

1. ¿Qué son los 'atributos de calidad' (quality attributes) en arquitectura de software?

- a) Los casos de uso funcionales que el sistema debe implementar según los requisitos del cliente. - b) Las métricas de cobertura de código obtenidas durante las pruebas unitarias. - c) El conjunto de librerías de terceros utilizadas en el proyecto. - d) Propiedades no funcionales del sistema, como rendimiento, seguridad, disponibilidad y mantenibilidad, que determinan qué tan bien el sistema cumple sus objetivos operativos. - **Respuesta:** (d) - **Explicación:** Los atributos de calidad (también llamados '-ilities': escalabilidad, disponibilidad, mantenibilidad, seguridad, testabilidad, etc.) son requisitos no funcionales que restringen cómo debe comportarse el sistema, a diferencia de los requisitos funcionales que dicen qué debe hacer. La arquitectura existe principalmente para satisfacer estos atributos: por ejemplo, si la disponibilidad es crítica, la arquitectura debe incorporar redundancia; si la escalabilidad lo es, debe permitir el crecimiento horizontal. Como suelen entrar en conflicto entre sí (más seguridad puede reducir el rendimiento), gran parte del trabajo arquitectónico consiste en negociar trade-offs explícitos entre ellos.

1. ¿Quiénes son los 'stakeholders' (interesados) de la arquitectura de software?

- a) Todas las personas o grupos con interés legítimo en el sistema: desarrolladores, usuarios

finales, gerentes de producto, operaciones, seguridad y clientes, entre otros. - b) Únicamente los desarrolladores que escriben el código del sistema. - c) Solamente los inversionistas que financian el proyecto de software. - d) Exclusivamente el equipo de control de calidad (QA) que prueba el sistema. - **Respuesta:** (a) - **Explicación:** Los stakeholders son cualquier persona cuyos intereses se ven afectados por la arquitectura del sistema o que puede influir en las decisiones sobre ella: usuarios finales (preocupados por usabilidad y rendimiento), operaciones (preocupada por la capacidad de despliegue y monitoreo), seguridad (preocupada por vulnerabilidades), gerencia (preocupada por costos y plazos), y desarrolladores (preocupados por mantenibilidad). El arquitecto de software actúa como mediador entre estos intereses, a menudo contrapuestos, documentando las decisiones y sus razones para que todos los stakeholders puedan entender por qué se eligió una solución sobre otra.

1. ¿Qué caracteriza a la arquitectura en capas (layered architecture)?

- a) Divide el sistema en servicios independientes que se comunican exclusivamente mediante colas de mensajes. - b) Organiza el sistema en capas horizontales (por ejemplo, presentación, lógica de negocio y acceso a datos), donde cada capa solo se comunica con las capas adyacentes. - c) Organiza el código alrededor de eventos que disparan reacciones en múltiples componentes desacoplados. - d) Estructura el sistema como un conjunto de plugins que se cargan dinámicamente en tiempo de ejecución. - **Respuesta:** (b) - **Explicación:** La arquitectura en capas (o N-tier) es uno de los estilos más antiguos y ampliamente utilizados. Organiza el sistema en niveles horizontales —típicamente presentación, lógica de negocio, y acceso a datos— donde cada capa expone servicios a la capa superior y consume servicios de la capa inferior, promoviendo separación de responsabilidades y facilidad de comprensión. Su principal ventaja es la simplicidad y el aislamiento de cambios (modificar la capa de datos no debería afectar la presentación); su principal desventaja es que puede introducir el 'sinkhole anti-pattern' (llamadas que atraviesan capas sin lógica real) y dificultar la escalabilidad independiente de cada capa.

1. En el estilo cliente-servidor, ¿cuál es la responsabilidad típica del servidor?

- a) Renderizar exclusivamente la interfaz gráfica que el usuario final visualiza. - b) Proveer recursos, datos o servicios que uno o más clientes consumen mediante solicitudes a través de una red. - c) Ejecutar únicamente pruebas automatizadas sobre el código del cliente. - d) Almacenar en caché local las preferencias del usuario sin exponer ninguna API. - **Respuesta:** (b) - **Explicación:** En el modelo cliente-servidor, el servidor centraliza recursos (datos, lógica de negocio, servicios) y responde a solicitudes de múltiples clientes, mientras que el cliente inicia las solicitudes y típicamente maneja la presentación. Este estilo simplifica la gestión y el control centralizado de los datos, pero introduce dependencia de la disponibilidad del servidor y puede convertirse en un cuello de botella si no se escala adecuadamente (por ejemplo, mediante balanceadores de carga o réplicas).

1. ¿Cuál es una diferencia clave entre una arquitectura monolítica y una de microservicios?

- a) Los microservicios siempre son más simples de operar que un monolito porque tienen menos partes móviles. - b) Un monolito no puede tener una base de datos relacional, a diferencia de los microservicios. - c) Los microservicios eliminan por completo la necesidad de pruebas de integración. - d) El monolito despliega toda la aplicación como una sola unidad, mientras que los microservicios despliegan servicios independientes que pueden escalar y evolucionar por separado. - **Respuesta:** (d) - **Explicación:** Un monolito empaqueta toda la funcionalidad de la aplicación en un único artefacto desplegable, lo que simplifica el desarrollo inicial y las pruebas, pero acopla el ciclo de vida de todos sus módulos. Los microservicios dividen el sistema en servicios pequeños y autónomos, cada uno con su propio ciclo de despliegue, escalado y, a menudo, su propia base de datos, lo que permite mayor flexibilidad organizacional (equipos independientes) a cambio de mayor complejidad operativa: hay que gestionar comunicación en red, consistencia de datos distribuidos, observabilidad y despliegues coordinados.

1. ¿Qué establece el principio de responsabilidad única (Single Responsibility Principle, la 'S' de SOLID)?

- a) Una clase debe implementar tantas interfaces como sea posible para maximizar la reutilización. - b) Todos los métodos de una clase deben ser estáticos para evitar el uso de instancias. - c) Un módulo debe depender directamente de las implementaciones concretas de sus colaboradores. - d) Una clase o módulo debe tener una única razón para cambiar, es decir, una sola responsabilidad bien definida. - **Respuesta:** (d) - **Explicación:** El SRP indica que cada módulo, clase o función debe encapsular una única responsabilidad o motivo de cambio dentro del sistema. Cuando una clase mezcla varias responsabilidades (por ejemplo, lógica de negocio y persistencia), un cambio en una de ellas puede romper la otra, y las pruebas se vuelven más difíciles de aislar. A nivel arquitectónico, este principio se traduce en la definición de límites claros entre módulos o servicios, cada uno enfocado en un dominio o capacidad de negocio específica, lo cual reduce el acoplamiento y facilita el mantenimiento a largo plazo.

1. ¿Qué establece el principio de abierto/cerrado (Open/Closed Principle, la 'O' de SOLID)?

- a) Todas las clases deben ser finales (final) para impedir cualquier tipo de herencia. - b) El código debe reescribirse por completo cada vez que se agrega una nueva funcionalidad. - c) Las entidades de software deben estar abiertas a extensión pero cerradas a modificación. - d) Las interfaces deben evitarse en favor de clases concretas para simplificar el diseño. - **Respuesta:** (c) - **Explicación:** El OCP propone que se debe poder añadir nuevo comportamiento a un sistema sin modificar el código existente que ya funciona y ha sido probado, típicamente mediante abstracciones (interfaces, clases abstractas) y polimorfismo. Esto reduce el riesgo de introducir regresiones al extender funcionalidad, ya que el código antiguo permanece intacto. A nivel arquitectónico, este principio se refleja en patrones como plugins, estrategias (Strategy

pattern) o puntos de extensión bien definidos, que permiten evolucionar el sistema agregando módulos nuevos en lugar de tocar los existentes.

1. ¿Qué describe mejor el concepto de 'acoplamiento' (coupling) entre módulos?

- a) La cantidad de líneas de código que contiene un módulo. - b) El grado de interdependencia entre dos módulos, donde un acoplamiento alto significa que un cambio en un módulo probablemente obligará a cambiar el otro. - c) El número de pruebas unitarias asociadas a un módulo. - d) La velocidad de ejecución de un módulo comparada con otro módulo similar. - **Respuesta:** (b) - **Explicación:** El acoplamiento mide cuánto depende un módulo de los detalles internos de otro. Un acoplamiento bajo (loose coupling) es deseable porque permite modificar, probar o reemplazar un módulo sin afectar a los demás, mientras que un acoplamiento alto (tight coupling) genera un efecto dominó de cambios y dificulta la evolución independiente del sistema. Las arquitecturas modernas favorecen el bajo acoplamiento mediante interfaces bien definidas, contratos explícitos (como APIs) y comunicación asíncrona que minimizan las dependencias directas entre componentes.

1. ¿Qué es la 'cohesión' en el contexto del diseño de módulos?

- a) El grado en que los elementos dentro de un mismo módulo están relacionados funcionalmente entre sí y contribuyen a un propósito común. - b) La cantidad de dependencias externas que utiliza un módulo. - c) El número de desarrolladores que trabajan simultáneamente en un módulo. - d) El tiempo que tarda un módulo en compilarse dentro del pipeline de CI. - **Respuesta:** (a) - **Explicación:** La cohesión mide qué tan enfocado y coherente es un módulo en términos de su propósito: un módulo altamente cohesivo agrupa funcionalidad estrechamente relacionada, lo que facilita entenderlo, probarlo y mantenerlo. El objetivo arquitectónico ideal es maximizar la cohesión dentro de cada módulo y minimizar el acoplamiento entre módulos ('alta cohesión, bajo acoplamiento'), ya que esta combinación produce sistemas más fáciles de modificar, pues los cambios tienden a quedar contenidos dentro de un único módulo en lugar de propagarse por todo el sistema.

1. ¿Qué busca el principio de separación de responsabilidades (Separation of Concerns)?

- a) Combinar toda la lógica de negocio y de presentación en un único archivo para simplificar el despliegue. - b) Eliminar la necesidad de definir interfaces entre los distintos módulos del sistema. - c) Dividir un sistema en secciones distintas de modo que cada una aborde una preocupación (concern) separada, minimizando la superposición de responsabilidades. - d) Garantizar que todos los módulos compartan el mismo espacio de memoria en tiempo de ejecución. - **Respuesta:** (c) - **Explicación:** La separación de responsabilidades es un principio general de diseño que busca dividir el software en partes distintas, cada una encargada de un aspecto específico del problema (por ejemplo, la persistencia, la validación, la

presentación o la seguridad), de forma que los cambios en una preocupación tengan un impacto mínimo en las demás. Este principio subyace a la mayoría de los estilos arquitectónicos (capas, hexagonal, microservicios) y es la base conceptual detrás de patrones como MVC, donde el modelo, la vista y el controlador se mantienen deliberadamente separados.

1. En una API REST, ¿qué caracteriza una operación 'idempotente'?

- a) La operación siempre devuelve una respuesta distinta cada vez que se invoca. - b) La operación solo puede ejecutarse una única vez durante todo el ciclo de vida del sistema. - c) Ejecutar la misma operación múltiples veces produce el mismo efecto que ejecutarla una sola vez. - d) La operación requiere autenticación obligatoria mediante OAuth2. - **Respuesta:** (c) - **Explicación:** La idempotencia es una propiedad clave en el diseño de APIs y sistemas distribuidos: una operación idempotente puede repetirse de forma segura sin causar efectos secundarios adicionales, lo cual es esencial cuando existen reintentos automáticos por fallos de red. Por ejemplo, PUT y DELETE en REST se consideran idempotentes (aplicar el mismo PUT dos veces deja el recurso en el mismo estado), mientras que POST normalmente no lo es (dos POST podrían crear dos recursos distintos). Diseñar operaciones idempotentes facilita la construcción de sistemas resilientes que pueden reintentar solicitudes sin temor a duplicar efectos.

1. ¿Cuál es una diferencia fundamental entre bases de datos relacionales y NoSQL?

- a) Las bases de datos NoSQL no pueden almacenar más de un millón de registros en total. - b) Las bases de datos relacionales no permiten realizar copias de seguridad (backups). - c) NoSQL significa que la base de datos no utiliza SQL bajo ninguna circunstancia, sin excepciones. - d) Las relacionales imponen un esquema fijo y relaciones mediante claves foráneas con fuertes garantías ACID, mientras que muchas NoSQL ofrecen esquemas flexibles priorizando escalabilidad horizontal. - **Respuesta:** (d) - **Explicación:** Las bases de datos relacionales organizan los datos en tablas con esquemas estrictos y relaciones definidas mediante claves foráneas, ofreciendo transacciones ACID robustas ideales para datos altamente estructurados y consistentes. Las bases de datos NoSQL (documentales, de clave-valor, de grafos o de columnas anchas) suelen relajar estas garantías o el esquema fijo a cambio de mayor escalabilidad horizontal y flexibilidad en el modelo de datos, lo cual las hace atractivas para grandes volúmenes de datos semi-estructurados o cargas de trabajo distribuidas. La elección entre ambas depende de los requisitos de consistencia, la forma de los datos y el patrón de acceso esperado.

1. ¿Qué patrón implementa el Modelo-Vista-Controlador (MVC)?

- a) Combina la lógica de negocio y la persistencia de datos en un único componente llamado 'modelo unificado'. - b) Elimina la necesidad de una capa de presentación al generar la interfaz

automáticamente desde la base de datos. - c) Separa la aplicación en tres componentes: el modelo (datos y lógica de negocio), la vista (presentación) y el controlador (coordina la entrada del usuario y actualiza el modelo y la vista). - d) Define que toda la lógica de la aplicación debe residir exclusivamente en el controlador. - **Respuesta:** (c) - **Explicación:** MVC es un patrón arquitectónico clásico para aplicaciones interactivas que separa las responsabilidades en tres partes: el Modelo mantiene el estado y las reglas de negocio, la Vista se encarga de representar esos datos al usuario, y el Controlador recibe las acciones del usuario, actualiza el modelo y selecciona la vista adecuada. Esta separación permite modificar la interfaz sin tocar la lógica de negocio y viceversa, y ha inspirado variantes como MVP y MVVM utilizadas ampliamente en frameworks web y de escritorio.

1. ¿Qué diferencia existe entre escalabilidad vertical y horizontal?

- a) La escalabilidad vertical solo aplica a bases de datos, mientras que la horizontal solo aplica a servidores web. - b) La escalabilidad horizontal requiere detener el sistema por completo, mientras que la vertical no. - c) Ambas son equivalentes y no existe ninguna diferencia práctica entre ellas. - d) La escalabilidad vertical añade más recursos (CPU, memoria) a una misma máquina, mientras que la horizontal añade más instancias o nodos que trabajan en conjunto. - **Respuesta:** (d) - **Explicación:** Escalar verticalmente (scale up) significa aumentar la capacidad de un único nodo —más CPU, RAM o almacenamiento— lo cual es sencillo de implementar pero tiene un límite físico y no mejora la tolerancia a fallos. Escalar horizontalmente (scale out) implica añadir más instancias o nodos que comparten la carga de trabajo, generalmente mediante un balanceador de carga, lo que mejora tanto la capacidad como la disponibilidad (si un nodo falla, otros continúan sirviendo tráfico), aunque introduce complejidad adicional relacionada con el estado compartido y la coordinación entre nodos.

1. ¿Qué significa que un sistema tenga 'alta disponibilidad' (high availability)?

- a) El sistema permanece operativo y accesible durante un porcentaje muy alto del tiempo, minimizando el tiempo de inactividad (downtime). - b) El sistema nunca necesita mantenimiento ni actualizaciones de ningún tipo. - c) El sistema procesa todas las solicitudes en menos de un milisegundo sin excepción. - d) El sistema utiliza exclusivamente una única instancia para simplificar su operación. - **Respuesta:** (a) - **Explicación:** La alta disponibilidad se refiere a diseñar el sistema para que continúe funcionando correctamente ante fallos parciales, minimizando el tiempo en que no está disponible para los usuarios, comúnmente medido en 'nueves' (99.9%, 99.99%, etc.). Se logra mediante redundancia (múltiples instancias de cada componente crítico), eliminación de puntos únicos de falla, detección automática de fallos y mecanismos de recuperación (failover). Es distinto de la tolerancia a fallos o de la durabilidad de los datos, aunque estos conceptos suelen combinarse en el diseño de sistemas críticos.

1. ¿Para qué se utilizan los diagramas de arquitectura (por ejemplo, el modelo C4)?

- a) Para reemplazar por completo la necesidad de escribir código fuente documentado. - b) Para comunicar visualmente la estructura del sistema en distintos niveles de abstracción (contexto, contenedores, componentes y código) a diferentes audiencias. - c) Para generar automáticamente las pruebas unitarias del sistema. - d) Para calcular el costo exacto de infraestructura en la nube. - **Respuesta:** (b) - **Explicación:** Los diagramas de arquitectura, como los del modelo C4 (Context, Containers, Components, Code), permiten comunicar la estructura de un sistema a distintos niveles de zoom, adaptando el nivel de detalle según la audiencia: un diagrama de contexto sirve para stakeholders no técnicos, mientras que un diagrama de componentes es útil para desarrolladores. Documentar la arquitectura visualmente facilita el onboarding de nuevos integrantes, la toma de decisiones informada y la detección temprana de problemas de diseño, aunque estos diagramas deben mantenerse actualizados para no volverse engañosos con el tiempo.

1. ¿Qué es un 'punto único de falla' (single point of failure, SPOF)?

- a) Un componente que nunca falla bajo ninguna circunstancia debido a su diseño robusto. - b) Un módulo que solo se ejecuta una vez durante el arranque del sistema. - c) Una prueba unitaria que cubre únicamente un caso límite del sistema. - d) Un componente cuyo fallo provoca la caída de todo el sistema o de una parte crítica de él, al no existir redundancia que lo respalde. - **Respuesta:** (d) - **Explicación:** Un SPOF es cualquier elemento del sistema —un servidor, una base de datos, un servicio de red— cuya falla individual interrumpe la operación completa porque no existe una alternativa o réplica que tome su lugar. Identificar y eliminar los SPOF es una de las tareas centrales del diseño de sistemas de alta disponibilidad, típicamente introduciendo redundancia (réplicas activas o pasivas), balanceo de carga y mecanismos de conmutación por error (failover) automáticos para que la caída de un componente no se traduzca en una interrupción total del servicio.

1. ¿Qué describe mejor el concepto de arquitectura de tres capas (three-tier architecture)?

- a) Divide físicamente el sistema en tres niveles: presentación, lógica de negocio y datos, que suelen desplegarse en servidores o procesos separados. - b) Es un sinónimo exacto de arquitectura de microservicios sin ninguna diferencia conceptual. - c) Requiere que las tres capas se ejecuten obligatoriamente en el mismo proceso y máquina física. - d) Elimina la necesidad de una base de datos al almacenar todo en memoria caché. - **Respuesta:** (a) - **E xplicación:** La arquitectura de tres capas es una especialización física de la arquitectura en capas, donde la capa de presentación, la capa de lógica de negocio y la capa de datos se despliegan típicamente en servidores o procesos distintos, comunicándose a través de la red. Esto permite escalar y actualizar cada capa de forma relativamente independiente y es la base de muchas aplicaciones empresariales tradicionales (por ejemplo, un frontend web, un servidor de aplicaciones y un servidor de base de datos separados), aunque introduce latencia de red

entre capas que no existe en un despliegue monolítico de proceso único.

Intermedio: Estilos y Patrones Arquitectónicos

1. ¿Qué caracteriza a la arquitectura orientada a eventos (Event-Driven Architecture, EDA)?

- a) Todos los componentes deben invocarse mediante llamadas síncronas RPC para garantizar el orden de ejecución. - b) Los componentes se comunican produciendo y reaccionando a eventos de forma asíncrona y desacoplada, sin que el productor conozca a los consumidores. - c) Requiere que exista una única base de datos compartida entre todos los servicios del sistema. - d) Sustituye por completo la necesidad de utilizar APIs REST en cualquier escenario. - **Respuesta:** (b) - **Explicación:** En EDA, los componentes (productores) publican eventos que representan hechos ocurridos en el sistema (por ejemplo, 'PedidoCreado') sin conocer quién los consumirá, y otros componentes (consumidores) reaccionan a esos eventos de forma independiente y asíncrona, típicamente a través de un broker de mensajes o un bus de eventos. Este desacoplamiento temporal y espacial mejora la escalabilidad y la resiliencia (un consumidor caído no bloquea al productor), pero introduce complejidad en la trazabilidad, el orden de los eventos y la consistencia eventual entre los distintos componentes del sistema.

1. ¿Cuál es el objetivo principal de la arquitectura hexagonal (Ports & Adapters)?

- a) Aislar la lógica de negocio del núcleo de la aplicación de los detalles externos (UI, bases de datos, APIs) mediante puertos e interfaces bien definidas. - b) Forzar a que toda la aplicación se comunique únicamente mediante seis módulos, uno por cada lado del hexágono. - c) Eliminar por completo la necesidad de pruebas de integración en el sistema. - d) Garantizar que la base de datos sea siempre el componente central del que dependen todos los demás módulos. - **Respuesta:** (a) - **Explicación:** La arquitectura hexagonal, propuesta por Alistair Cockburn, coloca la lógica de negocio (el dominio) en el centro del sistema, aislada mediante 'puertos' (interfaces que definen cómo el núcleo interactúa con el exterior) y 'adaptadores' (implementaciones concretas de esos puertos, como un adaptador REST o uno de base de datos). Esto permite que los detalles de infraestructura sean intercambiables sin afectar la lógica de negocio central: se puede cambiar una base de datos relacional por una NoSQL, o exponer la aplicación vía REST o mensajería, simplemente escribiendo un nuevo adaptador. El principal beneficio es la testabilidad del núcleo de negocio de forma aislada de la infraestructura.

1. ¿Qué resuelve el patrón CQRS (Command Query Responsibility Segregation)?

- a) Combina en un único modelo tanto las operaciones de lectura como las de escritura para simplificar el código. - b) Elimina la necesidad de una base de datos al mantener todo el estado únicamente en memoria. - c) Garantiza que todas las consultas se ejecuten de forma síncrona con cada escritura, sin excepción. - d) Separa los modelos utilizados para escribir datos (comandos) de los modelos utilizados para leer datos (consultas), permitiendo optimizar cada uno de forma independiente. - **Respuesta:** (d) - **Explicación:** CQRS propone utilizar modelos separados para las operaciones de escritura (comandos, que modifican el estado) y las operaciones de lectura (consultas, que solo devuelven datos), en lugar de un único modelo compartido para ambas. Esto permite optimizar independientemente cada lado: por ejemplo, el modelo de lectura puede ser una vista desnormalizada optimizada para consultas rápidas, mientras que el modelo de escritura se centra en mantener la integridad de las reglas de negocio. CQRS suele combinarse con Event Sourcing, pero introduce el reto de la consistencia eventual entre el modelo de escritura y las vistas de lectura, por lo que no conviene aplicarlo indiscriminadamente a todo el sistema.

1. ¿En qué consiste el patrón Event Sourcing?

- a) En eliminar por completo la base de datos y almacenar todo el estado en la memoria caché del cliente. - b) En almacenar el estado de una entidad como una secuencia de eventos inmutables que representan los cambios ocurridos, en lugar de guardar únicamente el estado actual. - c) En sobrescribir directamente el estado actual de la entidad cada vez que ocurre un cambio, sin conservar historial. - d) En replicar de forma síncrona cada tabla de la base de datos entre todos los microservicios del sistema. - **Respuesta:** (b) - **Explicación:** En Event Sourcing, en lugar de persistir únicamente el estado final de una entidad (por ejemplo, el saldo actual de una cuenta), se almacena la secuencia completa de eventos que llevaron a ese estado (Depósito, Retiro, Depósito...). El estado actual se puede reconstruir en cualquier momento reproduciendo esos eventos, lo cual ofrece una auditoría completa y natural del historial de cambios, facilita la depuración y permite reconstruir vistas alternativas del mismo historial. La contrapartida es la complejidad adicional: hay que gestionar el versionado de eventos, snapshots para reconstrucciones eficientes, y a menudo se combina con CQRS para separar la escritura de eventos de la lectura de vistas materializadas.

1. En Domain-Driven Design (DDD), ¿qué es un 'Bounded Context'?

- a) La totalidad del sistema de software, sin ninguna subdivisión interna. - b) Un límite físico entre centros de datos utilizados para el despliegue geográfico. - c) Un límite explícito dentro del cual un modelo de dominio específico es válido y consistente, con su propio lenguaje ubicuo. - d) El conjunto de todas las pruebas automatizadas de un microservicio. - **Respuesta:** (c) - **Explicación:** Un Bounded Context define la frontera dentro de la cual un modelo de dominio particular tiene un significado consistente y un lenguaje ubicuo compartido por el equipo. Un mismo término (por ejemplo, 'Cliente') puede tener significados y atributos distintos en diferentes contextos (el contexto de Ventas versus el contexto de Soporte), y DDD reconoce esto

explícitamente en lugar de forzar un modelo único y monolítico para todo el negocio. Los Bounded Contexts suelen alinearse con los límites de los microservicios, ya que cada contexto puede evolucionar, desplegarse y ser propiedad de un equipo de forma relativamente independiente.

1. En DDD, ¿qué es un 'Aggregate' (agregado)?

- a) Una tabla de base de datos que almacena únicamente valores numéricos agregados como sumas o promedios. - b) Un grupo de objetos de dominio relacionados que se tratan como una unidad consistente, con una raíz (Aggregate Root) que controla el acceso y garantiza las invariantes. - c) Un microservicio completo que agrupa todas las funcionalidades del sistema. - d) Un conjunto de pruebas de integración ejecutadas en paralelo durante el pipeline de CI/CD. - **Respuesta:** (b) - **Explicación:** Un Aggregate es un clúster de entidades y value objects que deben mantenerse consistentes entre sí según las reglas de negocio, y que se modifican siempre a través de su Aggregate Root, el único punto de entrada permitido desde el exterior. Por ejemplo, un 'Pedido' con sus 'Líneas de Pedido' puede formar un agregado donde el Pedido es la raíz: no se permite modificar una línea directamente sin pasar por el Pedido, que garantiza que las invariantes (como el total del pedido) permanezcan correctas. Definir agregados correctamente —ni demasiado grandes ni demasiado pequeños— es clave para el rendimiento y la consistencia transaccional en sistemas basados en DDD.

1. ¿Qué problema resuelve el patrón Circuit Breaker (disyuntor)?

- a) Evita que un sistema siga realizando llamadas repetidas a un servicio que está fallando, permitiendo que se recupere y evitando fallos en cascada. - b) Aumenta automáticamente la cantidad de instancias de un servicio cuando el tráfico crece. - c) Cifra el tráfico de red entre dos microservicios para mejorar la seguridad. - d) Convierte llamadas síncronas en llamadas asíncronas de forma transparente. - **Respuesta:** (a) - **Explicación:** El patrón Circuit Breaker monitorea las llamadas a un servicio externo o dependencia, y cuando detecta una tasa de fallos por encima de un umbral, 'abre el circuito' y deja de intentar la llamada durante un periodo, devolviendo un error rápido (fail fast) o una respuesta alternativa en lugar de esperar timeouts repetidos. Después de un tiempo, entra en un estado 'semi-abierto' donde permite pasar algunas solicitudes de prueba para verificar si el servicio se recuperó, y si es así, 'cierra el circuito' reanudando el tráfico normal. Este patrón es esencial en sistemas distribuidos para evitar que el fallo de un servicio se propague en cascada y agote los recursos (hilos, conexiones) de los servicios que dependen de él.

1. ¿Qué problema resuelve el patrón Saga en microservicios?

- a) Cifrar las comunicaciones entre microservicios utilizando certificados TLS mutuos. - b) Reducir la latencia de red entre servicios ubicados en distintas regiones geográficas. - c)

Mantener la consistencia de datos a través de múltiples servicios en una transacción de negocio de larga duración, sin usar una transacción distribuida ACID tradicional. - d) Eliminar por completo la necesidad de manejar errores en llamadas entre servicios. - **Respuesta:** (c) - **Explicación:** Cuando una operación de negocio abarca varios microservicios, cada uno con su propia base de datos, no es viable usar una transacción ACID distribuida clásica (como two-phase commit) por razones de escalabilidad y disponibilidad. El patrón Saga descompone la operación en una secuencia de transacciones locales, cada una publicando un evento que dispara el siguiente paso; si un paso falla, se ejecutan transacciones compensatorias que deshacen los pasos anteriores ya completados. Existen dos variantes principales: coreografía (los servicios reaccionan a eventos sin un coordinador central) y orquestación (un componente central dirige explícitamente la secuencia de pasos).

1. ¿Cuál es la función principal de un API Gateway en una arquitectura de microservicios?

- a) Almacenar de forma permanente todos los datos de negocio del sistema en un único lugar.
- b) Actuar como punto de entrada único que enruta las solicitudes a los servicios internos, centralizando tareas transversales como autenticación, rate limiting y agregación de respuestas.
- c) Reemplazar la necesidad de tener una base de datos en cada microservicio. - d) Ejecutar directamente la lógica de negocio de todos los microservicios del sistema. - **Respuesta:** (b) - **Explicación:** Un API Gateway centraliza el acceso externo a un conjunto de microservicios, actuando como fachada que oculta la complejidad interna del sistema a los clientes. Además de enrutar solicitudes al servicio correspondiente, típicamente maneja responsabilidades transversales como autenticación y autorización, limitación de tasa (rate limiting), agregación de múltiples respuestas de servicios en una sola, transformación de protocolos, y registro de métricas. Esto evita duplicar esa lógica en cada microservicio individual, aunque el gateway debe diseñarse cuidadosamente para no convertirse en un cuello de botella o un punto único de falla.

1. ¿Qué es un Service Mesh (malla de servicios)?

- a) Un tipo de base de datos NoSQL optimizada para almacenar relaciones entre entidades.
- b) Un lenguaje de programación diseñado específicamente para escribir microservicios. - c) Una herramienta de diseño gráfico utilizada para dibujar diagramas de arquitectura. - d) Una capa de infraestructura dedicada que gestiona la comunicación entre servicios (descubrimiento, balanceo, cifrado, reintentos, observabilidad) de forma transparente mediante proxies sidecar. - **Respuesta:** (d) - **Explicación:** Un Service Mesh (como Istio o Linkerd) despliega un proxy 'sidecar' junto a cada instancia de servicio, que intercepta todo el tráfico de red entrante y saliente. Esto permite implementar de forma centralizada y consistente funcionalidades como descubrimiento de servicios, balanceo de carga, cifrado mutuo TLS, reintentos, circuit breaking y recolección de métricas de observabilidad, sin que el código de cada microservicio tenga que implementar esa lógica individualmente. La contrapartida es la complejidad operativa adicional y la sobrecarga de latencia que introduce la capa de proxies, por lo que su adopción suele reservarse para arquitecturas con un número considerable de microservicios.

1. ¿Cuál es la principal diferencia entre comunicación síncrona y asíncrona entre microservicios?

- a) La comunicación asíncrona solo puede implementarse usando bases de datos compartidas entre servicios. - b) En la comunicación síncrona el emisor espera una respuesta inmediata (por ejemplo, HTTP/REST), mientras que en la asíncrona el emisor continúa sin esperar, típicamente a través de colas o eventos. - c) La comunicación síncrona nunca puede usarse en sistemas que requieren alta disponibilidad. - d) Ambas formas de comunicación producen exactamente los mismos niveles de acoplamiento temporal. - **Respuesta:** (b) - **Explicación:** En la comunicación síncrona (como llamadas HTTP/REST o gRPC), el servicio que realiza la solicitud queda bloqueado esperando la respuesta, lo que crea un acoplamiento temporal: si el servicio destino está caído o es lento, el llamador se ve directamente afectado. En la comunicación asíncrona (mediante colas de mensajes o brokers de eventos), el emisor publica un mensaje y continúa su ejecución sin esperar una respuesta inmediata, lo que reduce el acoplamiento temporal y mejora la resiliencia ante fallos parciales, a costa de una mayor complejidad para razonar sobre el flujo y el orden de procesamiento, así como la necesidad de manejar consistencia eventual.

1. ¿Cuál es una diferencia entre los algoritmos de balanceo de carga 'Round Robin' y 'Least Connections'?

- a) Round Robin solo puede utilizarse en bases de datos, nunca en servidores web. - b) Round Robin distribuye solicitudes de forma secuencial entre los servidores disponibles, mientras que Least Connections las envía al servidor con menos conexiones activas en ese momento. - c) Least Connections ignora por completo el estado de los servidores backend al distribuir tráfico. - d) Ambos algoritmos garantizan un rendimiento idéntico sin importar la carga de cada servidor. - **Respuesta:** (b) - **Explicación:** Round Robin es un algoritmo simple que asigna cada nueva solicitud al siguiente servidor en una lista circular, sin tener en cuenta la carga actual de cada uno, lo cual funciona bien cuando todos los servidores tienen capacidad similar y las solicitudes son homogéneas en costo. Least Connections, en cambio, dirige el tráfico al servidor que actualmente tiene menos conexiones activas, lo que resulta más eficaz cuando las solicitudes varían en duración o costo de procesamiento, evitando que un servidor sobrecargado siga recibiendo tráfico. La elección del algoritmo de balanceo adecuado depende del perfil de carga y de la homogeneidad de los servidores backend.

1. ¿Qué diferencia hay entre las estrategias de caché 'cache-aside' y 'write-through'?

- a) Cache-aside solo puede usarse con bases de datos NoSQL, mientras que write-through solo con relacionales. - b) Write-through elimina por completo la necesidad de una base de datos persistente. - c) En cache-aside la aplicación gestiona explícitamente la carga y actualización de la caché, mientras que en write-through cada escritura se realiza simultáneamente en la caché y en la base de datos. - d) Ambas estrategias son idénticas y

difieren únicamente en el nombre utilizado. - **Respuesta:** (c) - **Explicación:** En la estrategia cache-aside (o lazy loading), la aplicación primero consulta la caché; si no encuentra el dato (cache miss), lo carga desde la base de datos y lo almacena en la caché para futuras consultas, mientras que las escrituras suelen invalidar o actualizar la entrada correspondiente. En write-through, cada operación de escritura se propaga de forma síncrona tanto a la caché como a la base de datos, garantizando que la caché esté siempre actualizada a costa de una mayor latencia en cada escritura. La elección entre ambas depende del patrón de lectura/escritura de la aplicación: cache-aside es más común cuando las lecturas dominan y se tolera cierta latencia en el primer acceso, mientras que write-through favorece la consistencia inmediata entre caché y almacenamiento persistente.

1. ¿Qué diferencia principal existe entre una cola de mensajes (message queue) y un modelo publicador/suscriptor (pub/sub)?

- a) Las colas de mensajes no pueden garantizar el orden de los mensajes bajo ninguna circunstancia. - b) En una cola, cada mensaje normalmente es consumido por un único consumidor, mientras que en pub/sub un mismo evento puede ser entregado a múltiples suscriptores interesados. - c) El modelo pub/sub requiere obligatoriamente una base de datos relacional como backend. - d) Ambos modelos son exactamente equivalentes y se implementan siempre con la misma tecnología. - **Respuesta:** (b) - **Explicación:** En el modelo clásico de colas (point-to-point), cada mensaje se coloca en una cola y es consumido y eliminado por un único consumidor entre varios competidores, lo cual es útil para repartir trabajo (work queues) entre múltiples workers. En el modelo publicador/suscriptor, los eventos se publican en un tópico o canal, y todos los suscriptores interesados en ese tópico reciben una copia del mismo evento, lo cual es adecuado para difundir notificaciones o hechos de dominio a múltiples partes interesadas. Muchas plataformas modernas de mensajería (como Kafka o RabbitMQ) soportan ambos patrones de forma combinada según cómo se configuren los consumidores.

1. ¿Qué establece el teorema CAP sobre los sistemas distribuidos?

- a) Que todo sistema distribuido puede garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones sin ningún compromiso. - b) Que los sistemas distribuidos siempre deben priorizar el rendimiento sobre la seguridad. - c) Que las bases de datos relacionales nunca pueden distribuirse en múltiples nodos. - d) Que ante una partición de red, un sistema distribuido debe elegir entre garantizar consistencia o disponibilidad, no ambas al mismo tiempo. - **Respuesta:** (d) - **Explicación:** El teorema CAP (Brewer) afirma que un sistema distribuido no puede garantizar simultáneamente las tres propiedades de Consistencia (todos los nodos ven los mismos datos al mismo tiempo), Disponibilidad (toda solicitud recibe una respuesta, sin garantía de que sea la más reciente) y tolerancia a Particiones (el sistema sigue funcionando pese a fallos de comunicación entre nodos). Dado que las particiones de red son inevitables en sistemas distribuidos reales, en la práctica el teorema se

traduce en que, cuando ocurre una partición, hay que elegir entre priorizar la consistencia (rechazando solicitudes que no puedan garantizarla) o la disponibilidad (respondiendo aunque los datos puedan estar desactualizados). Esta decisión de diseño depende de los requisitos del negocio.

1. ¿Qué significa 'consistencia eventual' (eventual consistency)?

- a) Que, en ausencia de nuevas actualizaciones, todas las réplicas de un dato convergerán eventualmente al mismo valor, aunque puedan estar temporalmente desincronizadas. - b) Que los datos nunca se sincronizan entre las distintas réplicas del sistema. - c) Que cada lectura siempre devuelve el valor más reciente escrito, sin ninguna demora. - d) Que el sistema descarta automáticamente los datos más antiguos después de un breve periodo. - **Respuesta:** (a) - **Explicación:** La consistencia eventual es un modelo de consistencia más débil que la consistencia fuerte, común en sistemas distribuidos que priorizan disponibilidad y tolerancia a particiones (según el teorema CAP). Bajo este modelo, tras una escritura, distintas réplicas pueden devolver temporalmente valores diferentes, pero si no se producen nuevas escrituras, todas las réplicas convergerán eventualmente al mismo estado, normalmente mediante mecanismos de sincronización en segundo plano o resolución de conflictos. Este modelo es aceptable en muchos escenarios (como contadores de 'me gusta' en redes sociales) donde una pequeña demora en la sincronización no afecta significativamente la experiencia del usuario, pero no es adecuado para casos que requieren consistencia inmediata, como transacciones bancarias.

1. ¿En qué consiste el patrón Strangler Fig para modernizar sistemas legados?

- a) Reescribir todo el sistema legado desde cero en un único evento (big bang) antes de ponerlo en producción. - b) Ejecutar el sistema legado y el nuevo sistema de forma completamente aislada, sin ninguna interacción entre ambos. - c) Reemplazar gradualmente partes de un sistema legado con nuevos servicios, enrutando progresivamente el tráfico hacia la nueva implementación hasta retirar por completo el sistema antiguo. - d) Eliminar inmediatamente el sistema legado sin migrar ninguna funcionalidad previamente. - **Respuesta:** (c) - **Explicación:** El patrón Strangler Fig (nombrado por la higuera estranguladora, que crece alrededor de un árbol hasta reemplazarlo) propone modernizar un sistema legado de forma incremental en lugar de una reescritura completa arriesgada. Se introduce una fachada o proxy que enruta las solicitudes: inicialmente casi todo el tráfico va al sistema antiguo, y a medida que se desarrollan nuevas funcionalidades o módulos equivalentes, el tráfico correspondiente se redirige gradualmente hacia el nuevo sistema, hasta que el legado queda completamente reemplazado y puede retirarse. Esta estrategia reduce el riesgo al permitir validar cada pieza migrada de forma incremental y revertir cambios si algo falla, en lugar de apostar todo a una migración de una sola vez.

1. ¿Cuál es una diferencia clave entre contenedores y máquinas virtuales?

- a) Las máquinas virtuales siempre arrancan más rápido que los contenedores en cualquier escenario. - b) Los contenedores requieren obligatoriamente un hipervisor de tipo 1 para funcionar. - c) Los contenedores comparten el kernel del sistema operativo anfitrión y son más ligeros, mientras que las máquinas virtuales incluyen un sistema operativo completo con su propio kernel. - d) No existe ninguna diferencia relevante en cuanto a uso de recursos entre ambas tecnologías. - **Respuesta:** (c) - **Explicación:** Una máquina virtual virtualiza el hardware completo mediante un hipervisor y ejecuta un sistema operativo invitado completo (con su propio kernel), lo que ofrece fuerte aislamiento pero consume más recursos y tarda más en iniciar. Un contenedor, en cambio, virtualiza a nivel del sistema operativo, compartiendo el kernel del host pero aislando procesos, sistema de archivos y red mediante namespaces y cgroups, lo que lo hace mucho más ligero y rápido de iniciar, permitiendo desplegar muchos más contenedores por servidor físico. Esta ligereza ha impulsado la adopción de contenedores en arquitecturas de microservicios, aunque las VMs siguen siendo preferibles cuando se requiere aislamiento más fuerte a nivel de kernel o soporte multi-sistema operativo.

1. En Kubernetes, ¿qué es un 'Pod'?

- a) Un tipo de base de datos distribuida integrada de forma nativa en Kubernetes. - b) El nombre que recibe cada nodo físico o virtual dentro de un clúster de Kubernetes. - c) Un archivo de configuración YAML que define las políticas de seguridad de red del clúster. - d) La unidad mínima de despliegue, que agrupa uno o más contenedores que comparten red y almacenamiento y se ejecutan siempre juntos en el mismo nodo. - **Respuesta:** (d) - **Explicación:** Un Pod es la unidad más pequeña que Kubernetes puede desplegar y gestionar; agrupa uno o más contenedores estrechamente relacionados (por ejemplo, un contenedor principal y un sidecar de logging) que comparten el mismo espacio de red (misma IP) y, opcionalmente, volúmenes de almacenamiento. Los Pods son efímeros por diseño: Kubernetes los crea y destruye según sea necesario, y su gestión de ciclo de vida, escalado y actualización se delega típicamente a controladores de nivel superior como Deployments, StatefulSets o Jobs, que garantizan que exista siempre el número deseado de réplicas de un Pod en ejecución.

1. ¿Cuál es una característica distintiva de la arquitectura Serverless (FaaS)?

- a) El desarrollador debe aprovisionar y administrar manualmente cada servidor donde se ejecutará la función. - b) El código debe permanecer siempre en ejecución de forma continua, incluso sin recibir ninguna solicitud. - c) El proveedor de la nube gestiona automáticamente el aprovisionamiento y escalado de la infraestructura, y el código se ejecuta únicamente en respuesta a eventos, cobrándose por el tiempo de ejecución real. - d) Serverless significa literalmente que no existe ningún servidor físico en ningún lugar de la infraestructura del proveedor. - **Respuesta:** (c) - **Explicación:** En un modelo Serverless (o Function as a Service, FaaS), el desarrollador se enfoca únicamente en escribir funciones que se ejecutan en

respuesta a eventos (una solicitud HTTP, un mensaje en una cola, un cambio en una base de datos), mientras el proveedor de la nube gestiona de forma transparente el aprovisionamiento, escalado automático (incluyendo escalar a cero cuando no hay tráfico) y el cobro se basa en el tiempo de cómputo real utilizado, no en instancias reservadas permanentemente. A pesar del nombre, sí existen servidores detrás, pero su gestión queda completamente oculta al desarrollador. Este modelo reduce la carga operativa y es ideal para cargas de trabajo esporádicas o con picos variables, aunque introduce retos como el 'cold start' (latencia inicial al arrancar una función inactiva) y límites en la duración máxima de ejecución.

Avanzado: Arquitectura Avanzada y Sistemas Distribuidos

1. En el contexto del teorema CAP, ¿qué trade-off representa un sistema clasificado como 'AP' (Availability + Partition tolerance)?

- a) Prioriza seguir respondiendo a las solicitudes incluso durante una partición de red, aceptando que algunas respuestas puedan reflejar datos no completamente consistentes. - b) Prioriza rechazar todas las solicitudes durante una partición de red para garantizar consistencia absoluta. - c) Garantiza consistencia fuerte y disponibilidad total simultáneamente, incluso durante particiones de red. - d) Elimina por completo la posibilidad de que ocurra una partición de red en el sistema. - **Respuesta:** (a) - **Explicación:** Un sistema 'AP' —como muchas bases de datos NoSQL orientadas a alta disponibilidad (por ejemplo, Cassandra en su configuración típica)— elige mantenerse disponible y seguir respondiendo solicitudes de lectura y escritura incluso cuando la red está particionada, a costa de permitir que distintos nodos temporalmente devuelvan datos inconsistentes entre sí, los cuales convergen más tarde bajo consistencia eventual. Esto contrasta con un sistema 'CP', que preferiría rechazar solicitudes (sacrificando disponibilidad) antes que arriesgarse a devolver datos inconsistentes durante la partición. La elección entre AP y CP debe alinearse con los requisitos reales del negocio: un sistema de inventario crítico puede requerir CP, mientras que un sistema de recomendaciones puede tolerar AP.

1. ¿Qué problema resuelve el 'consistent hashing' (hashing consistente) en sistemas distribuidos?

- a) Garantizar que todas las claves se almacenen siempre en un único nodo central sin distribución alguna. - b) Cifrar las claves almacenadas en una base de datos distribuida para cumplir con normativas de seguridad. - c) Eliminar por completo la necesidad de particionar los datos entre distintos nodos del clúster. - d) Minimizar la cantidad de claves que deben reubicarse cuando se añaden o eliminan nodos de un clúster distribuido, en comparación con el

hashing tradicional por módulo. - **Respuesta:** (d) - **Explicación:** Con un hashing tradicional (clave módulo número de nodos), agregar o quitar un solo nodo cambia el resultado del módulo para la mayoría de las claves, forzando una redistribución masiva de datos entre nodos. El consistent hashing resuelve esto mapeando tanto los nodos como las claves a posiciones en un anillo circular de hash: cada clave se asigna al primer nodo encontrado en sentido horario en el anillo, de modo que añadir o quitar un nodo solo afecta a las claves adyacentes a ese nodo en el anillo, minimizando drásticamente la cantidad de datos que deben moverse. Esta técnica es fundamental en sistemas como Cassandra, DynamoDB y en el balanceo de caché distribuida (por ejemplo, Memcached).

1. ¿Qué es el patrón Sidecar en arquitecturas de microservicios?

- a) Ejecutar dos versiones completas del mismo microservicio en paralelo para pruebas A/B. - b) Almacenar copias de seguridad de la base de datos en un servidor secundario geográficamente distante. - c) Dividir un microservicio monolítico en dos microservicios completamente independientes. - d) Desplegar un proceso auxiliar junto al servicio principal, en el mismo host o Pod, para proveerle funcionalidades transversales (proxy, logging, métricas) sin modificar su código. - **Respuesta:** (d) - **Explicación:** El patrón Sidecar despliega un contenedor o proceso auxiliar adjunto al servicio principal (por ejemplo, en el mismo Pod de Kubernetes), que le proporciona capacidades adicionales —como un proxy de red, recolección de métricas, logging centralizado, o terminación TLS— sin necesidad de modificar el código del servicio principal ni acoplar esas responsabilidades transversales a su lógica de negocio. Este patrón es la base de la implementación de los Service Meshes (como Istio, que inyecta un sidecar Envoy junto a cada servicio), permitiendo estandarizar y actualizar esas capacidades de infraestructura de forma centralizada e independiente del ciclo de vida de cada aplicación.

1. ¿Qué es el 'backpressure' en sistemas de procesamiento de flujos (streaming)?

- a) Un mecanismo mediante el cual un consumidor más lento comunica al productor que reduzca la tasa de envío de datos, evitando saturar sus recursos. - b) Un tipo de fallo de red que ocurre exclusivamente en centros de datos ubicados en distintas zonas geográficas. - c) Una técnica de cifrado utilizada para proteger los datos que fluyen entre microservicios. - d) Un algoritmo de compresión aplicado a los mensajes antes de enviarlos por la red. - **Respuesta:** (a) - **Explicación:** El backpressure es un mecanismo de control de flujo esencial en sistemas reactivos y de procesamiento de streams, donde un productor puede generar datos más rápido de lo que un consumidor es capaz de procesar. Sin este mecanismo, el consumidor podría saturarse (por ejemplo, acumulando una cola ilimitada en memoria hasta agotar los recursos y colapsar). El backpressure permite que el consumidor señale explícitamente al productor cuánta capacidad tiene disponible, de modo que el productor ajuste su tasa de envío o el sistema aplique estrategias como buffering acotado, descarte de mensajes o bloqueo temporal del productor. Frameworks como Reactive Streams, RxJava o Akka Streams implementan este concepto de forma nativa.

1. ¿Cuál es la diferencia fundamental entre el protocolo de two-phase commit (2PC) y el patrón Saga para transacciones distribuidas?

- a) 2PC y Saga son exactamente el mismo mecanismo, solo que con nombres distintos en la literatura. - b) 2PC bloquea recursos hasta que todos los participantes confirmen (commit atómico y sincrónico), mientras que Saga usa transacciones locales independientes con compensaciones si algo falla, sin bloqueo global. - c) Saga requiere un coordinador central que bloquea todos los recursos hasta el final de la transacción. - d) 2PC nunca puede utilizarse en bases de datos relacionales, únicamente en sistemas NoSQL. - **Respuesta:** (b) - **Explicación:** El two-phase commit es un protocolo que garantiza atomicidad en transacciones distribuidas mediante una fase de 'preparación' (todos los participantes confirman que pueden comprometerse) y una fase de 'commit' (se aplican los cambios de forma coordinada), pero mantiene recursos bloqueados durante todo el proceso y depende de la disponibilidad continua de un coordinador, lo cual reduce la disponibilidad y no escala bien en sistemas altamente distribuidos. El patrón Saga, en cambio, evita el bloqueo global descomponiendo la transacción en pasos locales independientes, cada uno confirmado inmediatamente en su propio servicio; si un paso posterior falla, se ejecutan acciones compensatorias que deshacen lógicamente (no físicamente) los efectos de los pasos ya confirmados, sacrificando la atomicidad estricta a cambio de mayor disponibilidad y escalabilidad.

1. En Apache Kafka, ¿qué garantiza el particionamiento de un tópico respecto al orden de los mensajes?

- a) Kafka garantiza el orden global de todos los mensajes de un tópico, sin importar cuántas particiones existan. - b) El orden de los mensajes es completamente aleatorio y no puede controlarse de ninguna manera. - c) Kafka garantiza el orden de los mensajes únicamente dentro de una misma partición, no a través de todo el tópico. - d) El particionamiento en Kafka solo afecta al rendimiento, nunca al orden de los mensajes. - **Respuesta:** (c) - **Explicación:** Kafka distribuye los mensajes de un tópico entre múltiples particiones para permitir paralelismo en la escritura y lectura, pero solo garantiza el orden estricto de los mensajes dentro de una misma partición individual, no a través de todo el tópico. Los mensajes que comparten la misma clave (key) se enrutan de forma determinista a la misma partición (mediante hashing de la clave), lo que permite garantizar el orden relativo entre mensajes relacionados (por ejemplo, todos los eventos de un mismo usuario) eligiendo esa clave adecuadamente. Diseñar el esquema de particionamiento y la elección de claves es, por tanto, una decisión arquitectónica crítica que afecta tanto el paralelismo como las garantías de orden que el sistema puede ofrecer.

1. ¿Cuál es la diferencia principal entre un despliegue 'blue-green' y un despliegue 'canary'?

- a) Blue-green mantiene dos entornos completos y cambia todo el tráfico de una vez de uno a otro, mientras que canary libera la nueva versión gradualmente a un pequeño porcentaje de

usuarios antes de expandirla. - b) Blue-green y canary son términos distintos para describir exactamente la misma técnica de despliegue. - c) Canary requiere detener por completo el sistema durante el despliegue, a diferencia de blue-green. - d) Blue-green solo puede aplicarse a bases de datos, nunca a servicios de aplicación. - **Respuesta:** (a) - **Explicación:** En un despliegue blue-green se mantienen dos entornos de producción idénticos (blue y green); mientras uno sirve el tráfico en vivo, el otro se actualiza con la nueva versión, y una vez validado, el tráfico se conmuta completamente de un entorno al otro (por ejemplo, cambiando el balanceador de carga), permitiendo un rollback instantáneo si algo falla. En un despliegue canary, la nueva versión se libera inicialmente solo a un pequeño porcentaje del tráfico o usuarios reales, y se monitorean métricas clave (errores, latencia) antes de incrementar gradualmente el porcentaje hasta el 100%, lo que permite detectar problemas con impacto limitado antes de una liberación completa. Ambas estrategias buscan minimizar el riesgo de un despliegue, pero canary ofrece una validación progresiva más granular a costa de mayor complejidad de enrutamiento.

1. ¿Cuál es el objetivo principal de la Ingeniería del Caos (Chaos Engineering)?

- a) Provocar fallos aleatorios sin ningún control para medir el tiempo que tarda el equipo en detectar un incidente no anunciado. - b) Eliminar por completo cualquier posibilidad de fallo en el sistema mediante pruebas unitarias exhaustivas. - c) Inyectar fallos controlados y deliberados en un sistema en producción o en un entorno similar para verificar su resiliencia real ante condiciones adversas. - d) Sustituir las pruebas automatizadas tradicionales por experimentos manuales realizados por el equipo de QA. - **Respuesta:** (c) - **Explicación:** La Ingeniería del Caos, popularizada por Netflix con herramientas como Chaos Monkey, consiste en diseñar experimentos controlados que introducen fallos deliberados (apagar instancias, simular latencia de red, saturar CPU) en un sistema, con hipótesis claras sobre el comportamiento esperado, para verificar empíricamente que los mecanismos de resiliencia (redundancia, circuit breakers, auto-scaling) funcionan como se espera bajo condiciones adversas reales. A diferencia de las pruebas tradicionales que verifican comportamiento esperado bajo condiciones controladas, el chaos engineering busca descubrir debilidades desconocidas ('unknown unknowns') en sistemas distribuidos complejos, ejecutando los experimentos de forma progresiva, primero en entornos de prueba y eventualmente en producción con un 'blast radius' limitado.

1. ¿Qué es el 'cold start' en arquitecturas serverless y por qué es relevante?

- a) El tiempo que tarda un desarrollador en configurar por primera vez su cuenta en la nube. - b) Un error crítico que impide que una función serverless se ejecute permanentemente. - c) El proceso de apagar manualmente todas las funciones serverless al final del día. - d) La latencia adicional que ocurre cuando el proveedor debe inicializar una nueva instancia de la función (cargar el runtime y el código) porque no hay ninguna instancia 'caliente' disponible para atender la solicitud. - **Respuesta:** (d) - **Explicación:** En un entorno serverless, el proveedor

escala las funciones a cero cuando no reciben tráfico, liberando recursos; cuando llega una nueva solicitud y no existe una instancia ya inicializada ('warm'), el proveedor debe aprovisionar un nuevo contenedor o entorno de ejecución, cargar el runtime y el código de la función antes de poder procesar la solicitud, lo cual introduce una latencia adicional conocida como 'cold start' que puede variar desde decenas de milisegundos hasta varios segundos según el lenguaje y el tamaño del paquete. Esta latencia es especialmente problemática para aplicaciones sensibles a la latencia (como APIs sincrónicas de cara al usuario), y se mitiga mediante técnicas como mantener instancias 'calientes' con solicitudes periódicas (warm-up), reducir el tamaño del paquete desplegado, o elegir runtimes con arranque más rápido.

1. ¿Qué son los CRDTs (Conflict-free Replicated Data Types)?

- a) Un tipo de cifrado utilizado exclusivamente para proteger claves de bases de datos distribuidas. - b) Un algoritmo de compresión de datos aplicado antes de replicar información entre centros de datos. - c) Estructuras de datos diseñadas para replicarse en múltiples nodos y poder fusionarse automáticamente sin conflictos, sin necesidad de coordinación en tiempo real entre réplicas. - d) Un protocolo de autenticación utilizado para validar la identidad de nodos en un clúster. - **Respuesta:** (c) - **Explicación:** Los CRDTs son estructuras de datos matemáticamente diseñadas de forma que múltiples réplicas puedan actualizarse de forma independiente y concurrente (incluso sin conectividad entre sí), y luego fusionarse (merge) de manera determinista y libre de conflictos, garantizando que todas las réplicas converjan al mismo estado final sin necesidad de coordinación central ni bloqueos distribuidos. Ejemplos comunes incluyen contadores G-Counter (solo incremento), conjuntos OR-Set, o registros LWW-Register (last-write-wins). Son especialmente útiles en sistemas colaborativos en tiempo real, edición de documentos distribuida, y bases de datos multi-región que priorizan disponibilidad y tolerancia a particiones sobre consistencia fuerte inmediata.

1. ¿En qué se diferencia el modelo de seguridad 'Zero Trust' del modelo tradicional de seguridad perimetral?

- a) Zero Trust asume que ninguna solicitud es confiable por defecto, verificando explícitamente cada acceso sin importar si proviene de dentro o fuera de la red corporativa. - b) Zero Trust elimina por completo la necesidad de autenticación dentro de la red corporativa interna. - c) El modelo perimetral tradicional verifica cada solicitud individual, mientras que Zero Trust confía en la red interna. - d) Zero Trust solo aplica a bases de datos, nunca a servicios de aplicación o APIs. - **Respuesta:** (a) - **Explicación:** El modelo de seguridad perimetral tradicional asume que todo lo que está 'dentro' de la red corporativa (detrás del firewall) es confiable, concentrando las defensas en el perímetro. Zero Trust invierte esta suposición: ninguna solicitud, dispositivo o usuario se considera confiable por defecto, independientemente de si proviene de la red interna o externa, y cada acceso debe autenticarse y autorizarse explícitamente, aplicando el principio de mínimo privilegio y verificación continua (identidad, estado del dispositivo, contexto). Este modelo es particularmente relevante en

arquitecturas de microservicios y nube, donde el concepto de un perímetro fijo se diluye al distribuirse los servicios entre múltiples proveedores, regiones y redes.

1. ¿Qué desafío arquitectónico central debe resolver un sistema multi-tenant (multi-inquilino)?

- a) Garantizar que cada cliente tenga obligatoriamente su propio centro de datos físico dedicado. - b) Eliminar por completo la necesidad de autenticación para cada uno de los inquilinos del sistema. - c) Impedir que los distintos inquilinos puedan personalizar cualquier aspecto de la aplicación. - d) Aislar de forma segura los datos y recursos de cada cliente (tenant) mientras se comparte la misma infraestructura y código para reducir costos operativos. - **Respuesta:** (d) - **Explicación:** Una arquitectura multi-tenant permite que múltiples clientes (tenants) compartan la misma instancia de la aplicación y, a menudo, la misma infraestructura subyacente, lo cual reduce significativamente los costos operativos comparado con desplegar una instancia dedicada por cliente. El reto central es garantizar el aislamiento adecuado entre tenants —a nivel de datos (evitando que un tenant acceda a datos de otro), rendimiento (evitar que un tenant con carga alta degrade el servicio de otros, 'noisy neighbor') y configuración (permitir personalización por tenant)—, eligiendo entre estrategias como bases de datos separadas por tenant, esquemas separados dentro de una misma base de datos, o una única tabla compartida con una columna discriminadora de tenant, cada una con diferentes trade-offs entre aislamiento, costo y complejidad operativa.

1. ¿Qué propone el enfoque de Data Mesh para la arquitectura de datos en organizaciones grandes?

- a) Descentralizar la propiedad de los datos hacia los equipos de dominio que los generan, tratando los datos como un producto, en lugar de centralizarlos en un único equipo o plataforma de datos. - b) Centralizar absolutamente todos los datos de la organización en un único data warehouse gestionado por un solo equipo. - c) Eliminar por completo la necesidad de gobernanza de datos al distribuir la responsabilidad entre equipos. - d) Sustituir todas las bases de datos relacionales de la organización por un único sistema de archivos distribuido. - **Respuesta:** (a) - **Explicación:** Data Mesh es un enfoque organizacional y arquitectónico que responde a los cuellos de botella de los data warehouses o data lakes centralizados tradicionales, donde un equipo central de datos se convierte en un cuello de botella al intentar entender y gestionar los datos de todos los dominios de negocio. Propone descentralizar la propiedad de los datos hacia los equipos de dominio que mejor los conocen (los mismos que producen los microservicios), tratando cada conjunto de datos como un 'producto de datos' con calidad, documentación y SLAs definidos, expuesto mediante una infraestructura de autoservicio y gobernado por políticas federadas y computacionales en lugar de un control central rígido. Este enfoque busca escalar la gestión de datos en organizaciones grandes con muchos dominios independientes.

1. ¿Cuáles son los tres pilares clásicos de la observabilidad en sistemas distribuidos?

- a) Logs (registros), métricas (metrics) y trazas distribuidas (traces). - b) Backups, replicación y particionamiento de bases de datos. - c) Autenticación, autorización y auditoría de accesos. - d) Integración continua, despliegue continuo y control de versiones. - **Respuesta:** (a) - **Explicación:** La observabilidad en sistemas distribuidos se apoya tradicionalmente en tres tipos de telemetría complementarios: los logs son registros discretos y detallados de eventos específicos (útiles para depuración puntual), las métricas son valores numéricos agregados a lo largo del tiempo (como latencia promedio o tasa de errores, útiles para detectar tendencias y disparar alertas), y las trazas distribuidas siguen el recorrido completo de una solicitud a través de múltiples servicios, permitiendo identificar en qué componente específico ocurre un cuello de botella o un fallo. Herramientas como OpenTelemetry buscan estandarizar la recolección de estos tres pilares de forma unificada, ya que ninguno de los tres por sí solo ofrece visibilidad completa sobre el comportamiento de un sistema distribuido complejo.

1. ¿Qué previene el patrón Bulkhead (mamparo) en el diseño de sistemas resilientes?

- a) Que dos versiones distintas de un mismo servicio se ejecuten simultáneamente en producción. - b) Que el agotamiento de recursos (hilos, conexiones) causado por un componente que falla se propague y afecte a otros componentes del sistema, aislando los recursos en compartimentos independientes. - c) Que los datos de un microservicio sean accesibles directamente por otro microservicio sin pasar por una API. - d) Que un balanceador de carga distribuya el tráfico de manera desigual entre los servidores disponibles. - **Respuesta:** (b) - **Explicación:** El patrón Bulkhead, inspirado en los compartimentos estancos de un barco que evitan que una vía de agua hunda toda la embarcación, propone aislar los recursos (como pools de hilos, conexiones de red o memoria) asignados a distintas dependencias o funcionalidades del sistema, de modo que si una dependencia específica falla o se satura, solo agota los recursos de su propio compartimento y no los del resto del sistema. Por ejemplo, en lugar de usar un único pool de hilos compartido para todas las llamadas a servicios externos, se puede asignar un pool independiente por cada servicio dependiente, de forma que un servicio lento o caído no consuma todos los hilos disponibles y bloquee las llamadas a otros servicios sanos.

1. ¿Cuál es la diferencia principal entre la arquitectura Lambda y la arquitectura Kappa para procesamiento de datos masivos (big data)?

- a) Lambda solo puede procesar datos históricos, mientras que Kappa solo puede procesar datos en tiempo real, sin ninguna superposición. - b) Lambda mantiene dos rutas de procesamiento separadas (batch y streaming) que luego se combinan, mientras que Kappa procesa todo únicamente a través de una única ruta de streaming. - c) Kappa requiere obligatoriamente el doble de infraestructura que Lambda para funcionar correctamente. - d) Ambas arquitecturas son idénticas en su diseño, solo difieren en el nombre asignado por sus

creadores. - **Respuesta:** (b) - **Explicación:** La arquitectura Lambda combina dos rutas de procesamiento paralelas: una capa batch que procesa grandes volúmenes de datos históricos con alta precisión pero mayor latencia, y una capa de velocidad (speed layer) que procesa datos en tiempo real con menor latencia pero potencialmente menor precisión, combinando ambos resultados en una capa de servicio. Esto ofrece robustez pero implica mantener y sincronizar dos sistemas de procesamiento distintos con lógica potencialmente duplicada. La arquitectura Kappa, propuesta como simplificación, elimina la capa batch y procesa todos los datos —tanto históricos como en tiempo real— exclusivamente a través de una única ruta de streaming (reproduciendo el log completo de eventos cuando se necesita reprocesar datos históricos), reduciendo la complejidad operativa a costa de requerir que el motor de streaming sea capaz de manejar eficientemente grandes volúmenes de reprocesamiento.

1. ¿Cuál es el propósito principal del 'distributed tracing' (trazado distribuido) implementado por estándares como OpenTelemetry?

- a) Cifrar el tráfico de red entre distintos microservicios para cumplir con normativas de seguridad. - b) Reemplazar por completo la necesidad de recolectar métricas agregadas del sistema. - c) Generar automáticamente la documentación de la API de cada microservicio del sistema. - d) Seguir el recorrido completo de una solicitud individual a través de múltiples servicios, registrando la latencia y el contexto de cada segmento (span) para diagnosticar problemas de rendimiento. - **Respuesta:** (d) - **Explicación:** El distributed tracing asigna un identificador único (trace ID) a cada solicitud entrante que se propaga a través de todos los servicios que participan en su procesamiento, y cada servicio registra 'spans' que representan las unidades de trabajo realizadas (por ejemplo, una consulta a base de datos o una llamada a otro servicio), junto con su duración y metadatos relevantes. Al reconstruir la traza completa, un ingeniero puede visualizar exactamente por dónde pasó una solicitud, cuánto tiempo tomó cada segmento, y en qué componente específico ocurrió un cuello de botella o un error, lo cual sería extremadamente difícil de diagnosticar únicamente con logs aislados en sistemas con decenas o cientos de microservicios interconectados. OpenTelemetry es un estándar abierto que unifica la instrumentación de trazas, métricas y logs entre distintas herramientas y proveedores.

1. En el marco de Arquitectura Empresarial TOGAF, ¿qué representa el ADM (Architecture Development Method)?

- a) Un ciclo iterativo de fases que guía el proceso de desarrollo de la arquitectura empresarial, desde la visión y los requisitos hasta la implementación y la gestión del cambio. - b) Un lenguaje de programación especializado utilizado exclusivamente para definir arquitecturas empresariales. - c) Una base de datos centralizada que almacena todos los diagramas UML de una organización. - d) Un protocolo de red diseñado para la comunicación segura entre sistemas empresariales distribuidos. - **Respuesta:** (a) - **Explicación:** El ADM es el núcleo metodológico de TOGAF (The Open Group Architecture Framework), un ciclo iterativo compuesto por fases (Visión de la Arquitectura, Arquitectura de Negocio, de Sistemas de Información,

Tecnológica, Oportunidades y Soluciones, Planificación de la Migración, Gobernanza de la Implementación y Gestión de Cambios) que guía a las organizaciones en el desarrollo, implementación y mantenimiento de su arquitectura empresarial de forma estructurada y repetible. A diferencia de los marcos arquitectónicos centrados en un sistema individual, TOGAF y su ADM operan a nivel de toda la organización, alineando la estrategia de negocio con las capacidades tecnológicas, y suelen adoptarse en grandes empresas que necesitan coordinar múltiples iniciativas y sistemas interrelacionados de forma coherente.

1. ¿Qué papel cumplen las 'feature flags' (banderas de características) en una arquitectura de despliegue progresivo?

- a) Permiten activar o desactivar funcionalidades específicas en tiempo de ejecución sin necesidad de un nuevo despliegue, desacoplando el despliegue del código de la liberación de la funcionalidad al usuario. - b) Sirven exclusivamente para documentar qué funcionalidades están pendientes de desarrollo en el backlog. - c) Reemplazan por completo la necesidad de realizar pruebas automatizadas antes de un despliegue. - d) Solo pueden aplicarse a nivel de infraestructura, nunca a nivel de código de aplicación. - **Respuesta:** (a) - **Explicación:** Las feature flags son mecanismos de configuración que permiten activar o desactivar una funcionalidad en tiempo de ejecución, sin necesidad de desplegar nuevo código, lo cual desacopla el acto de 'desplegar' código (llevarlo a producción) del acto de 'liberar' una funcionalidad (hacerla visible a los usuarios). Esto habilita estrategias avanzadas como despliegues canary controlados por flag, pruebas A/B, activación gradual de funcionalidades por segmento de usuarios, y rollback instantáneo de una funcionalidad problemática simplemente desactivando el flag, sin necesidad de revertir un despliegue completo. Su uso extendido requiere disciplina para evitar la acumulación de 'deuda de flags' (código condicional obsoleto que nunca se limpia una vez que la funcionalidad se estabiliza).

1. ¿Qué es una 'fitness function' (función de aptitud) en el contexto de la arquitectura evolutiva (evolutionary architecture)?

- a) Una métrica de rendimiento físico calculada exclusivamente para servidores en centros de datos. - b) Un algoritmo de machine learning utilizado para predecir el crecimiento del equipo de desarrollo. - c) Una prueba automatizada y objetiva que verifica continuamente si la arquitectura mantiene un atributo de calidad deseado a medida que el sistema evoluciona. - d) Un documento estático que se redacta una única vez al inicio del proyecto y nunca se actualiza. - **Respuesta:** (c) - **Explicación:** En la arquitectura evolutiva, propuesta por autores como Neal Ford y Rebecca Parsons, una fitness function es un mecanismo automatizado (una prueba, un análisis estático, un chequeo en el pipeline de CI) que evalúa objetivamente si la arquitectura sigue cumpliendo un atributo de calidad definido como importante —por ejemplo, verificar que ningún módulo del dominio dependa directamente de un framework de infraestructura, o que la latencia de una API se mantenga bajo un umbral—, ejecutándose de forma continua en cada cambio del sistema. Este enfoque permite que la arquitectura evolucione incrementalmente de

forma guiada y protegida, detectando automáticamente el momento en que un cambio introduce una violación de una restricción arquitectónica importante, en lugar de depender únicamente de revisiones manuales esporádicas.

Cómo practicar

Este repositorio incluye una app de autoevaluación (self-test-app/) con dos simuladores de examen basados en architectural-interview-questions.json:

- `self-test-app/architecture_quiz_console.py` — quiz interactivo por terminal, con gráfico de resultados al finalizar.
- `self-test-app/architecture_quiz_gui.py` — quiz con interfaz gráfica (tkinter): quiz completo, quiz rápido, quiz personalizado, filtrado por categoría y modo estudio.

Consulta self-test-app/README.md para instrucciones de instalación y ejecución.